CSE 4304-Data Structures Lab. Winter 2022
Date: October 11, 2022.
Target Group: All
Topic: Binary Search Trees (BST)

Instructions:

- Task naming format: fullID_T01L05_2A.c/CPP
- If you find any issues in the problem description/test cases, comment in the google classroom.
- If you find any tricky test case that I didn't include but others might forget to handle, please comment! I'll be happy to add.
- Use appropriate comments in your code. This will help you to recall the solution in the future easily.
- Obtained marks will vary based on the efficiency of the solution.
- Do not use <bits/stdc++.h> library, add the necessary libraries one after another.
- Do not use any built-in functions or any STL libraries. If the solution is extremely difficult without the usage of such components, discuss it with the instructors.
- Modified sections will be marked with BLUE color.

Group	**Tasks**
1B	1 2 3
2B	1 2 4 5
2A	1 2 5 6
1A	1 2 4 6

Task-1:

A series of values are being inserted in a BST. Your task is to show the balance factor of every node after each insertion.

balance_factor= heightofLeftSubtree - heightofRightSubtree

The following requirements have to be addressed:

- Continue taking input until -1.
- After each insertion, print the nodes of the tree in an inorder fashion. Show the balance_factor beside each node within a bracket.
- Each node has the following attributes: data, left pointer, right pointer, parent pointer, and node height. (store balance_factor if needed)
- Your code should have the following functions:
    - Void Insertion (key) // iteratively inserts key
    - Void Update_height(node) // update the height of a node
    - Int height(node) // returns the height of a node
    - Int balance_factor(node) // returns balance factor

Sample Input	**Sample Output**
12 8 5 11 20 4 7 17 18 -1	12(0) 8(0) 12(1) 5(0) 8(1) 12(2) 5(0) 8(0) 11(0) 12(2) 5(0) 8(0) 11(0) 12(1) 20(0) 4(0) 5(1) 8(1) 11(0) 12(2) 20(0) 4(0) 5(0) 7(0) 8(1) 11(0) 12(2) 20(0) 4(0) 5(0) 7(0) 8(1) 11(0) 12(1) 17(0) 20(1) 4(0) 5(0) 7(0) 8(1) 11(0) 12(0) 17(-1) 18(0) 20(2)

Note:

- If the balance_factor isn't stored, showing it for every node might result in O(n) [it's allowed in this solution]. However, after insertion, height needs to be updated only for the ancestors O(**height**).

Task-2:
Utilize the functions implemented in Task-1 to provide a complete solution for maintaining a 'Balanced BST.'
The program will continue inserting values until it gets -1. For each insertion, it checks whether the newly inserted node has imbalanced any node or not. If any imbalanced node is found, 'rotation' is used to fix the issue.

Your program must include the following functions:

- Void insertion (key) // iteratively inserts key
- Void update_height(node) // update the height of a node
- Int height(node) // returns the height of a node
- Int balance_factor(node) // returns balance factor
- Left_rotate(node)
- Right_rotate(node)
- check_balance(node):
  - check if a node is imbalanced and calls relevant rotations.
- print_avl(root) // inorder traversal

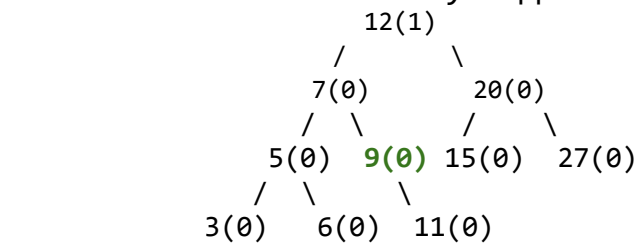
Sample Input	**Sample Output**
12	12(0) Balanced Root=12
9	**9**(0) 12(1) Balanced Root=12
5	5(0) **9**(1) 12(2) Imbalance at node: 12 LL case **right**_rotate(12) Status: 5(0) **9**(0) 12(0) Root=**9**
11	5(0) **9**(-1) 11(0) 12(1) Balanced Root=**9**
20	5(0) **9**(-1) 11(0) 12(**0**) 20(0) Balanced Root=**9**

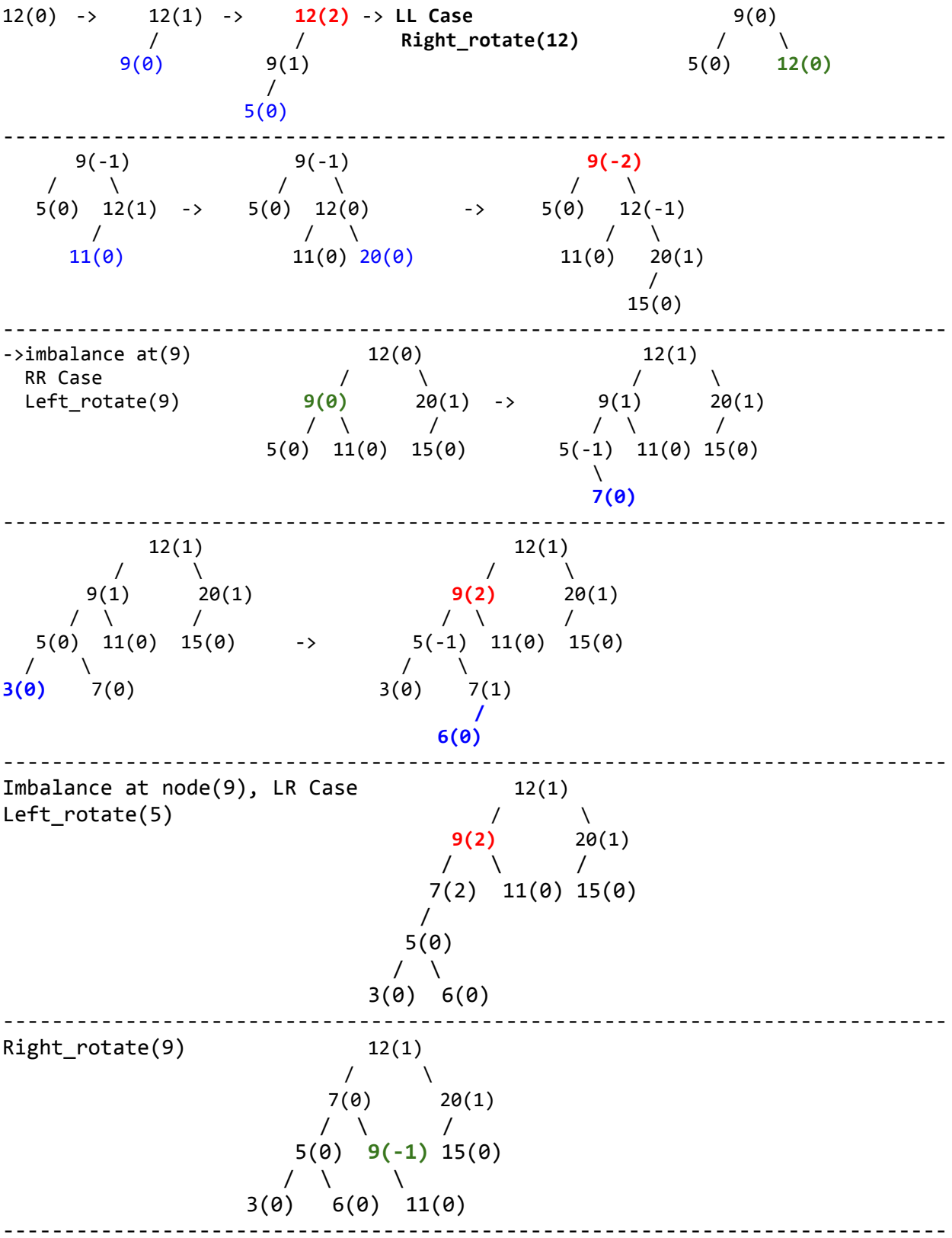
15	5(0) **9**(-2) 11(0) 12(-11) 15(0) 20(1) Imbalance at node: **9** RR case Left_rotate(**9**) Status: 5(0) **9**(0) 11(0) 12(0) 15(0) 20(1) Root=12
7	5(-1) 7(0) **9**(1) 11(0) 12(1) 15(0) 20(1) Balanced Root=12
3	3(0) 5(0) 7(0) **9**(1) 11(0) 12(1) 15(0) 20(1) Balanced Root=12
6	3(0) 5(-1) 6(0) 7(1) **9**(2) 11(0) 12(1) 15(0) 20(1) Imbalance at node: 9 LR Case Left_rotate(5), right_rotate(9) 3(0) 5(0) 6(0) 7(0) **9**(-1) 11(0) 12(1) 15(0) 20(1) Root=12
27	3(0) 5(0) 6(0) 7(0) **9**(-1) 11(0) 12(1) 15(0) 277(0) 20(0) Balanced Root=12
-1	Status: 3(0) 5(0) 6(0) 7(0) **9**(-1) 11(0) 12(1) 15(0) 20(0) 27(0)

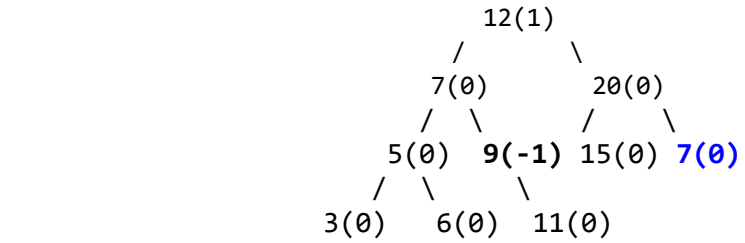
Note:

- **Do not** use any recursive implementation

The status of the tree is finally supposed to be like this:



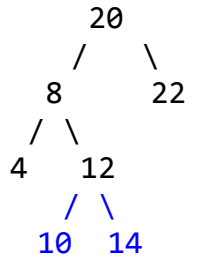

12(0) -> 12(1) -> 12(2) -> LL Case
Right_rotate(12)
9(0)
9(1)
5(0)
9(0)
5(0) 12(0)
9(-1)
5(0) 12(1) ->
11(0)
9(-1)
5(0) 12(0)
11(0) 20(0)
-> 9(-2)
5(0) 12(-1)
11(0) 20(1)
15(0)
->imbalance at(9)
RR Case
Left_rotate(9)
12(0)
9(0) 20(1) ->
5(0) 11(0) 15(0)
12(1)
9(1) 20(1)
5(-1) 11(0) 15(0)
7(0)
12(1)
9(1) 20(1)
5(0) 11(0) 15(0) ->
3(0) 7(0)
12(1)
9(2) 20(1)
5(-1) 11(0) 15(0)
3(0) 7(1)
6(0)
Imbalance at node(9), LR Case
Left_rotate(5)
12(1)
9(2) 20(1)
7(2) 11(0) 15(0)
5(0)
3(0) 6(0)
Right_rotate(9)
12(1)
7(0) 20(1)
5(0) 9(-1) 15(0)
3(0) 6(0) 11(0)


12(1)
7(0)
20(0)
5(0)
9(-1)
15(0)
7(0)
3(0)
6(0)
11(0)

Task-3:

Let T be a rooted tree. The **Lowest Common Ancestor**(LCA) between two nodes n1 and n2 is the lowest node in T with both n1 and n2 as descendants (we allow a node to be its descendant).
The LCA of n1 and n2 in T is the shared ancestor of n1 and n2 located farthest from the root. Computation of lowest common ancestors may be helpful, for instance, as part of a procedure for determining the distance between pairs of nodes in a tree.

Let's assume two nodes of a BST are n1 and n2. Your task is to find out their LCA. For example, suppose the status of the tree is as follows:



LCA of 10 & 14: 12
LCA of 4 & 22: 20
LCA of 4 & 8 : 8
LCA of 4 & 12 :8 (20 is also a common ancestor, but not 'lowest')
LCA of 8 & 14: 8
LCA of 8 & 22: 20
LCA of 20 & 8 : 20
LCA of 10 & 12: 12

Steps:

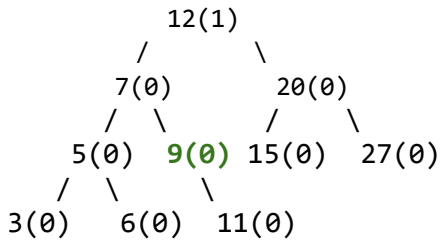
- At first, a sequence of N integers will be inserted (until -1)
- The numbers are inserted in an AVL Tree. The final status of the tree is shown after all nodes are inserted.
- Then there will be a number **T**, followed by T pairs of n1 & n2
- For each pair, your task is to print the LCA.

(PTO)

Sample Input	**Sample Output**
Input sequence 12 9 5 11 20 15 7 3 6 27 -1	Status: 3(0) 5(0) 6(0) 7(0) **9**(-1) 11(0) 12(1) 15(0) 20(0) 27(0)
11 3 6 6 11 27 9 12 27 27 12 6 7 9 3 7 20 20 15 11 7 11 27 -1	 5 7 12 12 12 7 **7** 12 20 7 12

Note:

- Use AVL Tree
- The solution must be O(**logN**)

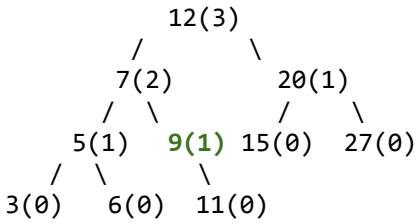


Task 4

Suppose a set of numbers is stored in a Balanced BST. Your task is to find the second largest element.

Input will be a sequence of numbers where the last value will be -1, denoting the end. The output will contain two lines. The first line will show the second largest element, and the next line will show the numbers in sorted fashion. Beside each number, show the height of the BST.

Sample Input	**Sample Output**
12 9 5 11 20 15 7 3 6 27 -1	20 Status: 3(0) 5(1) 6(0) 7(2) **9**(1) 11(0) 12(3) 15(0) 20(1) 27(0)
10 15 20 5 8 -1	15 Status: 5(0) 8(1) 10(0) 15(2) 20(0)

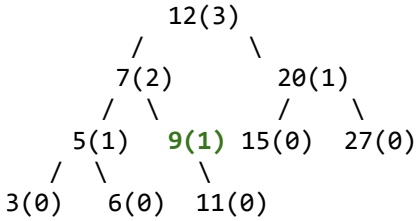


Task 5

A set of numbers is stored in a Balanced BST. Given two keys, your task is to print the path between the two nodes and the distance (number of nodes comprising that path).

At first, the numbers will be given as input. Then there will be a series of queries. Each query contains two numbers x & y (x<y). Print the path to reach ‘y’ starting from ‘x’. Then count the number of nodes in that path (include x,y).

Sample Input	**Sample Output**
12 9 5 11 20 15 7 3 6 27 -1	Status: 3(0) 5(1) 6(0) 7(2) **9**(1) 11(0) 12(3) 15(0) 20(1) 27(0)
5 11	5 7 9 11 4
3 12	3 5 7 12 4
6 11	6 5 7 9 11 5
7 9	7 9 2
7 11	7 9 11 3
3 15	3 5 7 12 20 15 6

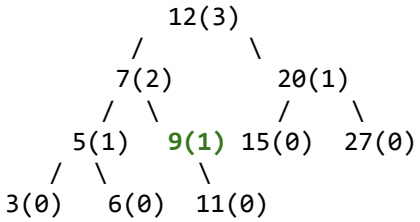


Task 6

A set of numbers is stored in a Balanced BST. Given two keys x & y representing a range, your task is to print all the items of the BST that lies within that range.

At first, the numbers will be given as input. Then there will be a series of queries. Each query contains two numbers x & y (x<y). Print all the values that lie within that range in a sorted fashion.

Sample Input	**Sample Output**
12 9 5 11 20 15 7 3 6 27 -1	Status: 3(0) 5(1) 6(0) 7(2) **9**(1) 11(0) 12(3) 15(0) 20(1) 27(0)
11 12	11 12
11 15	11 12 20
6 15	5 6 7 9 11 12 15
5 11	5 7 9 11
3 12	3 5 6 7 9 11 12
7 9	7 9
7 11	7 9 11
3 15	3 5 6 7 9 11 12 15
12 15	12 15
12 27	12 15 20 27



Note:

- Instead of blindly traversing the tree in inorder fashion, try to optimize the traversal as much as possible based on the range values.
- Finding the successor for each node starting from x also works. But it is even worse than in-order search.